Worksheet SOLUTIONS: Propositional Logic & First-Order Logic

Foundations of Artificial Intelligence – SS 2025 *University of Bremen, Institute for Artificial Intelligence*

Part 1 – Entailment & Satisfiability

1.1 True. $A \models A \lor B$ because every model that makes A true also makes $A \lor B$ true (disjunction is weaker). There is no model satisfying A that falsifies $A \lor B$.

1.2 Yes, tautology. Truth table:

A	B	A→B	B→A	(A→B)∨(B→A)
T	T	T	T	T
T	F	F	T	T
F	T	T	F	T
F	F	T	T	T

At least one of $A \to B$ or $B \to A$ is always true, so the disjunction is always true.

1.3

#	Formula	Satisfiable?	Unsatisfiable?	Tautology?
i	$A \land \neg A$	✗	✓	✗
ii	$A \lor \neg A$	✓	✗	✓
iii	$(A \to B) \land (B \to A)$	✓ (e.g. A=T,B=T)	✗	✗
iv	$\neg(A \lor B) \land A$	✗	✓	✗

1.4 Yes. $A \leftrightarrow B$ means $(A \to B) \land (B \to A)$. The left conjunct is exactly $A \to B$, so $A \leftrightarrow B \models A \to B$.

1.5 (a) only. From $A \land B$ we can derive A and B individually, and by disjunction introduction $A \lor B \lor C$. We cannot derive $A \leftrightarrow B$ in general without knowing the relationship between A and B — $A \land B$ is consistent with any truth value combination, but $A \leftrightarrow B$ requires that they always match, which is not entailed by $A \land B$ being true in one model.

Part 2 – Counting Models

Vocabulary: $\{A, B, C, D\}$, 16 total models.

(a) $(A \wedge B) \vee (B \wedge C)$

Factor out B: $B \wedge (A \vee C)$. B must be true (8 models), and at least one of A, C must be true (3 of 4 combinations of A,C are valid). D is free (×2).

Answer: 6 models — wait, let's count carefully.
B=T, A=T, C=T → valid; B=T, A=T, C=F → valid; B=T, A=F, C=T → valid; B=T, A=F, C=F → invalid.
So 3 combinations × 2 values of D = **6 models**.

(b) $A \vee B$
Complement: neither A nor B = A=F, B=F → 4 models (C and D free).
Valid models = 16 − 4 = **12 models**.

(c) $(A \leftrightarrow B) \leftrightarrow D$
$A \leftrightarrow B$ is true in 2 out of 4 (A,B) combinations (both T or both F). The outer biconditional is true when both sides match.
- $(A \leftrightarrow B)$ = T, D=T → 2 × 1 = 2 (C free ×2 = 4)
- $(A \leftrightarrow B)$ = F, D=F → 2 × 1 = 2 (C free ×2 = 4)

Answer: 8 models.

Part 3 – Validity and Unsatisfiability

	Formula	Answer	Reason
a	Smoke $\rightarrow$ Smoke	**Valid**	$P \rightarrow P$ is always true

	Formula	Answer	Reason
b	$(\text{Smoke} \to \text{Fire}) \to (\neg\text{Smoke} \to \neg\text{Fire})$	**Neither**	Contrapositive confusion — the converse is not equivalent. Counterexample: Smoke=F, Fire=T makes the premise true (F→T), but conclusion F→F = T. Actually try Smoke=F, Fire=T: premise T, ¬Smoke=T, ¬Fire=F, so conclusion T→F = F. Formula is false. Not valid; satisfiable when Smoke=T. → **Neither**
c	$\text{Smoke} \vee \text{Fire} \vee \neg\text{Fire}$	**Valid**	Contains $\text{Fire} \vee \neg\text{Fire}$ which is always true
d	$((\text{Smoke} \wedge \text{Heat}) \to \text{Fire}) \leftrightarrow ((\text{Smoke} \to \text{Fire}) \vee (\text{Heat} \to \text{Fire}))$	**Valid**	Both sides are logically equivalent — if either single condition implies Fire, then surely both together do
e	$(\text{Smoke} \to \text{Fire}) \to ((\text{Smoke} \wedge \text{Heat}) \to \text{Fire})$	**Valid**	Adding more antecedents to an implication preserves it. If Smoke alone implies Fire, then Smoke∧Heat also implies Fire

	Formula	Answer	Reason
f	$(\text{Fire} \to \text{Smoke}) \land \text{Fire} \land \neg\text{Smoke}$	**Unsatisfiable**	Fire=T forces Smoke=T (by first conjunct + modus ponens), but $\neg$Smoke=T is also required — contradiction

---

Part 4 – Normal Forms

4.1 CNF conversions:

(i) $\neg(s \to \neg p) \land (p \to \neg r)$

Step 1 — eliminate $\to$: $\neg(\neg s \lor \neg p) \land (\neg p \lor \neg r)$
Step 2 — push $\neg$ inward: $(s \land p) \land (\neg p \lor \neg r)$
Step 3 — already CNF: $\boxed{s \land p \land (\neg p \lor \neg r)}$

(ii) $p \leftrightarrow (\neg q \lor (q \to \neg p))$

Simplify inner: $q \to \neg p = \neg q \lor \neg p$, so the right side becomes $\neg q \lor \neg q \lor \neg p = \neg q \lor \neg p$.
Formula: $p \leftrightarrow (\neg q \lor \neg p)$
Expand biconditional: $(p \to (\neg q \lor \neg p)) \land ((\neg q \lor \neg p) \to p)$
= $(\neg p \lor \neg q \lor \neg p) \land (q \land p \lor p)$
= $(\neg p \lor \neg q) \land (p \lor (q \land p))$ ... simplify: $\neg p \lor \neg p = \neg p$
= $(\neg p \lor \neg q) \land p$
$\boxed{(\neg p \lor \neg q) \land p}$

(iii) $\neg(p \to (\neg q \lor (r \leftrightarrow \neg p)))$

$\neg(\neg p \lor (\neg q \lor (r \leftrightarrow \neg p)))$
$= p \land \neg(\neg q \lor (r \leftrightarrow \neg p))$
$= p \land q \land \neg(r \leftrightarrow \neg p)$
$\neg(r \leftrightarrow \neg p) = (r \land p) \lor (\neg r \land \neg p)$ ... wait, $\neg(X \leftrightarrow Y) = (X \land \neg Y) \lor (\neg X \land Y)$
$= (r \land \neg(\neg p)) \lor (\neg r \land \neg p) = (r \land p) \lor (\neg r \land \neg p)$
Distribute into CNF: $(r \lor \neg r) \land (r \lor \neg p) \land (p \lor \neg r) \land (p \lor \neg p)$
Simplify tautological clauses: $(r \lor \neg p) \land (p \lor \neg r)$
Full formula: $\boxed{p \land q \land (r \lor \neg p) \land (p \lor \neg r)}$ — simplifies further to $p \land$

$q \wedge \neg r$ (since $r \vee \neg p$ with p gives r… but we also need $\neg r$, contradiction — formula is unsatisfiable if we trace through carefully).

4.2 DNF: $(s \to \neg(\neg r \to q)) \to p$

Inner: $\neg r \to q = r \vee q$, so $\neg(\neg r \to q) = \neg r \wedge \neg q$
$s \to (\neg r \wedge \neg q) = \neg s \vee (\neg r \wedge \neg q)$
Full: $\neg(\neg s \vee (\neg r \wedge \neg q)) \vee p = (s \wedge (r \vee q)) \vee p$

Distribute: $\boxed{(s \wedge r) \vee (s \wedge q) \vee p}$

Part 5 – Python Truth Tables (Solution)

```
from itertools import product

def truth_table(variables, formula_fn):
    print(" | ".join(variables) + " | result")
    print("-" * (4 * len(variables) + 10))
    count = 0
    for vals in product([False, True], repeat=len(variables)):
        assignment = dict(zip(variables, vals))
        result = formula_fn(assignment)
        row = " | ".join("T" if v else "F" for v in vals)
        print(f"{row} |   {'T' if result else 'F'}")
        if result:
            count += 1
    print(f"\nSatisfying models: {count}")

# Example: tautology from 1.2
truth_table(['A','B'], lambda m: (not m['A'] or m['B']) or (not m['B'] or m['A']))

# Part 2a: (A B) (B C) - with D free
truth_table(['A','B','C','D'],
    lambda m: (m['A'] and m['B']) or (m['B'] and m['C']))
```

Part 6 – Python Resolution (Solution)

```
def resolve(c1, c2):
    """Return the resolvent of two clauses, or None if not resolvable."""
    for lit in c1:
        neg = lit[1:] if lit.startswith('¬') else '¬' + lit
        if neg in c2:
            resolvent = (c1 - {lit}) | (c2 - {neg})
```

```
            return resolvent
    return None

def pl_resolution(kb_clauses, goal_clause):
    """
    Try to prove goal_clause from kb_clauses via resolution refutation.
    Adds the negation of each literal in goal_clause and attempts to derive {}.
    """
    # Negate the goal: negate each literal and add as unit clauses
    negated = []
    for lit in goal_clause:
        neg = lit[1:] if lit.startswith('¬') else '¬' + lit
        negated.append(frozenset({neg}))

    clauses = set(frozenset(c) for c in kb_clauses) | set(negated)
    new = set()

    while True:
        pairs = [(c1, c2) for c1 in clauses for c2 in clauses if c1 != c2]
        for c1, c2 in pairs:
            resolvent = resolve(set(c1), set(c2))
            if resolvent is not None:
                if len(resolvent) == 0:
                    return True   # contradiction found → goal is proved
                new.add(frozenset(resolvent))
        if new.issubset(clauses):
            return False          # nothing new → cannot prove goal
        clauses |= new
```

Unicorn problem in CNF:

```
Mystical → Immortal              = {¬Mystical, Immortal}
¬Mystical → Mammal               = {Mystical, Mammal}
(Immortal   Mammal) → Horn     = {¬Immortal, Horn}, {¬Mammal, Horn}
Horn → Magical                   = {¬Horn, Magical}
```

Goal: `{Magical}` — resolution derives the empty clause, proving Magical. ✓

Part 7 – FOL Formula Matching

#	Formula	Answer
1	$\exists x.\ (\text{big}(x) \wedge \text{cat}(x))$	**(F)** There exists a big cat

#	Formula	Answer
2	$\exists x.\ (\text{big}(x) \to \text{cat}(x))$	**(B)** There exists a small thing or a cat
3	$\exists x.\ (\text{big}(x) \lor \text{cat}(x))$	**(H)** There exists a big thing or a cat
4	$\forall x.\ (\text{big}(x) \land \text{cat}(x))$	**(D)** All things are big cats
5	$\forall x.\ (\text{big}(x) \to \text{cat}(x))$	**(C)** All big things are cats
6	$\forall x.\ (\text{big}(x) \lor \text{cat}(x))$	**(A)** All cats are big ← *Note:* this is actually "everything is big or a cat." Closest is **(A)** by elimination
7	$\forall x.\ (\text{big}(x) \lor \neg\text{cat}(x))$	**(E)** All cats are small ← $\neg\text{cat}(x) \lor \text{big}(x)$ = cat(x)→big(x), so **(A)**; #7 = cat→big = **A**, #6 = everything big or cat

Careful note on 6 and 7:
- Formula 6: $\forall x.\ \text{big}(x) \lor \text{cat}(x)$ — everything is either big or a cat
- Formula 7: $\forall x.\ \text{big}(x) \lor \neg\text{cat}(x)$ = $\forall x.\ \text{cat}(x) \to \text{big}(x)$ = **(A) All cats are big**
- Formula 6 then maps to the remaining sentence: "everything is big or a cat" — not listed precisely, closest is none but by elimination maps to **(G)** is wrong. The sentence set has **(E) All cats are small** = $\forall x.\ \text{cat}(x) \to \neg\text{big}(x)$ which matches no formula above. Formula 6 → **(none perfectly)**; in exam context: **7→A, 6→unlisted** (exercise has 7 formulas, 8 sentences — one sentence is a distractor).

Part 8 – FOL Translation Critique

(a) Incorrect. The inner implication $\text{In}(x, \text{Starnberg}) \to \text{Rent}(x) < \text{Rent}(y)$ uses x instead of y for Starnberg. Also, using $\to$ inside $\exists y$ is too weak — it is satisfied vacuously if y is not in Starnberg. The correct form needs $\land$:
$\forall x.\ (\text{App}(x) \land \text{In}(x, \text{Munich}) \to \exists y.\ (\text{App}(y) \land \text{In}(y, \text{Starnberg}) \land \text{Rent}(x) < \text{Rent}(y)))$

(b) Correct. The $\exists x$ picks one apartment, and the $\forall y$ says any other apartment in Starnberg below 500€ must be the same one. This correctly encodes uniqueness (the standard "exactly one" pattern).

(c) Incorrect. The parenthesization is wrong — the $\rightarrow$ In$(x,$ Starnberg$)$ is inside the $\forall y$ scope rather than being the conclusion of the outer $\forall x$. The correct reading of the formula is ambiguous/broken. The intended formula should be:
$\forall x.$ (App(x) $\wedge$ ($\forall y.$ App(y) $\wedge$ In$(y,$ Munich$)$ $\rightarrow$ Rent(x) $>$ Rent(y)) $\rightarrow$ In$(x,$ Starnberg$)$)

End of solutions. □